

CAPACITAÇÃO EM DESENVOLVIMENTO FULL STACK

MÓDULO 09: Banco de Dados e SQL

Carga Horária: 32 horas

Professor: Janei Vieira Pereira



SUMÁRIO

Aula 1 — Introdução a Bancos de Dados	4
Conceitos básicos	4
Importância dos bancos de dados	4
Modelos de dados	4
Aula 2 — Arquitetura de SGBD e Modelo Relacional	7
Arquitetura básica de um SGBD	7
Tipos de SGBD	7
Entidades, atributos e relacionamentos	7
Aula 3 — Chaves, Integridade, Normalização e DER	9
Chave primária e chave estrangeira	9
Integridade referencial	9
Normalização	9
DER	10
Aula 4 — SQL DDL	11
CREATE, ALTER e DROP	11
Aula 5 — SQL DML, WHERE e ORDER BY	12
INSERT, SELECT, UPDATE e DELETE	12
WHERE e ORDER BY	12
Aula 6 — GROUP BY, HAVING e Funções Agregadas	13
Agrupamentos e agregações	13
Aula 7 — JOINS e Subconsultas	14
Joins	14
Subconsultas	14
Aula 8 — Recursos Avançados de SQL	15
Views	15
Índices	15
Transações	15
Procedures, funções e triggers	15
Aula 9 — Projeto prático e revisão	16
Projeto de banco relacional	16
Consultas complexas	16

Referências bibliográficas	17
Bibliografia básica	17
Bibliografia complementar.....	17

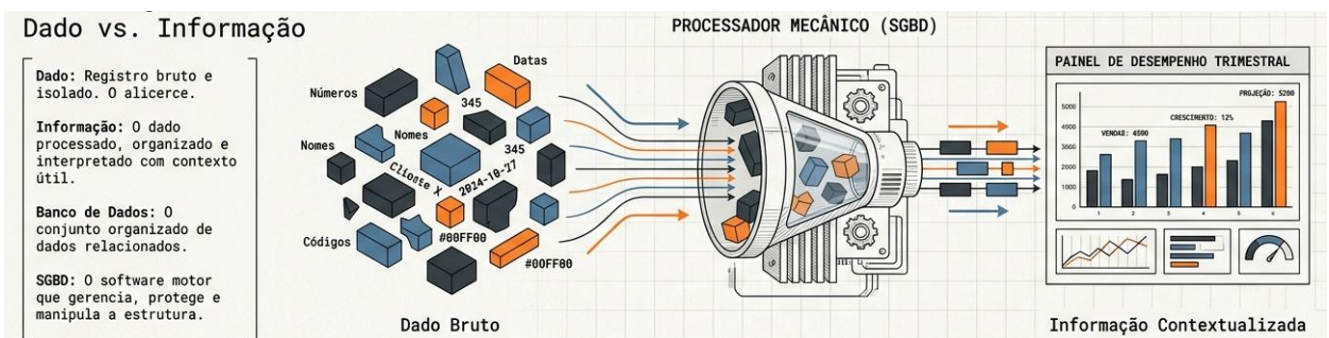
AULA 1 — INTRODUÇÃO A BANCOS DE DADOS

Conceitos básicos

Dados são registros brutos, como números, nomes, datas e códigos. Informação é o resultado da organização e interpretação desses dados em um contexto útil.

Banco de dados é um conjunto organizado de dados relacionados. O SGBD é o software responsável por armazenar, manipular, consultar, proteger e administrar esses dados.

Figura 1: fluxo visual mostrando dado bruto, processamento e informação gerada.



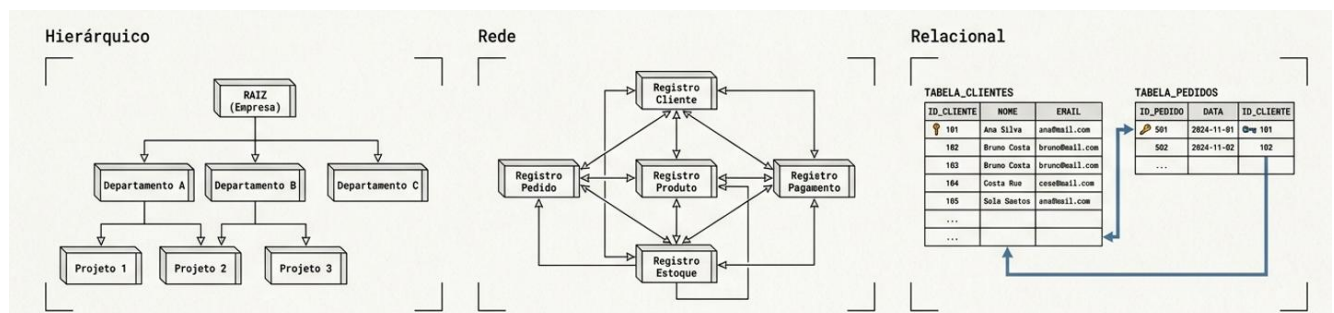
Importância dos bancos de dados

Os bancos de dados são essenciais em sistemas web, sistemas acadêmicos, e-commerces, aplicativos de cadastro, APIs e plataformas corporativas. Eles ajudam a evitar redundância, melhorar a integridade e facilitar o acesso às informações.

Modelos de dados

- Modelo hierárquico: estrutura em árvore.
- Modelo em rede: relacionamentos mais flexíveis.
- Modelo relacional: dados organizados em tabelas.

Figura 2: Modelos de Bancos de Dados



Para compreendermos o valor das tecnologias que vamos dominar neste curso, precisamos primeiro olhar para o passado e entender como a organização da informação evoluiu na história da computação. Antigamente, as informações dos sistemas eram armazenadas em arquivos de texto simples ou planilhas.

Essa abordagem funcionava apenas em cenários muito pequenos. À medida que os sistemas e as empresas cresceram, essa limitação cobrou seu preço. Começaram a surgir problemas graves com o grande volume de arquivos, a falta de consistência das informações e os conflitos gerados por múltiplos usuários tentando acessar e modificar dados ao mesmo tempo.

Para resolver a complexidade do armazenamento, nasceram os Sistemas Gerenciadores de Bancos de Dados (SGBD) e os primeiros modelos de estruturação de dados. Essa evolução passou por três fases principais:

Modelo Hierárquico: Foi a primeira grande tentativa de organização. Nele, os dados são estruturados no formato de uma árvore, estabelecendo uma relação estrita de "pai para filho", onde um registro filho depende unicamente de um elemento pai. Embora funcionasse para cenários muito rígidos, esse modelo tinha pouca flexibilidade para representar relações mais complexas do mundo real.

Modelo em Rede: Para contornar as limitações estruturais da árvore, foi criado o modelo em rede. Ele permitia múltiplas conexões entre registros, formando uma espécie de grafo.

Apesar de resolver o problema da flexibilidade, ele esbarrou em outro obstáculo: a teia de relacionamentos físicos tornava o banco de dados extremamente complexo para os desenvolvedores entenderem e manterem a longo prazo.

Modelo Relacional: Por fim, o modelo relacional surgiu propondo uma quebra de paradigma. Ele abandonou a navegação por conexões e ponteiros físicos complexos e passou a organizar os dados em tabelas (compostas por linhas como registros e colunas como atributos). Em vez de caminhos rígidos, as tabelas passaram a ser conectadas por associações lógicas através do uso de chaves primárias e chaves estrangeiras.

Por que o modelo relacional venceu e se tornou o padrão? O modelo relacional se consolidou no mercado e é o mais utilizado até os dias de hoje porque uniu o melhor dos dois mundos: robustez e acessibilidade. Ele venceu a corrida tecnológica pelos seguintes motivos:

Simplicidade: Sua estrutura baseada em tabelas é muito mais simples de visualizar e entender do que árvores ou grafos complexos.

Linguagem Padronizada (SQL): Trouxe facilidade imensa de consulta e manipulação através de comandos SQL.

Solidez: Conta com uma forte base teórica (álgebra relacional), o que garante integridade aos dados.

Adoção em Massa: Esses benefícios geraram uma ampla adoção no mercado, impulsionando a criação de SGBDs poderosos e populares que utilizamos hoje, como MySQL, PostgreSQL, Oracle Database e SQL Server.

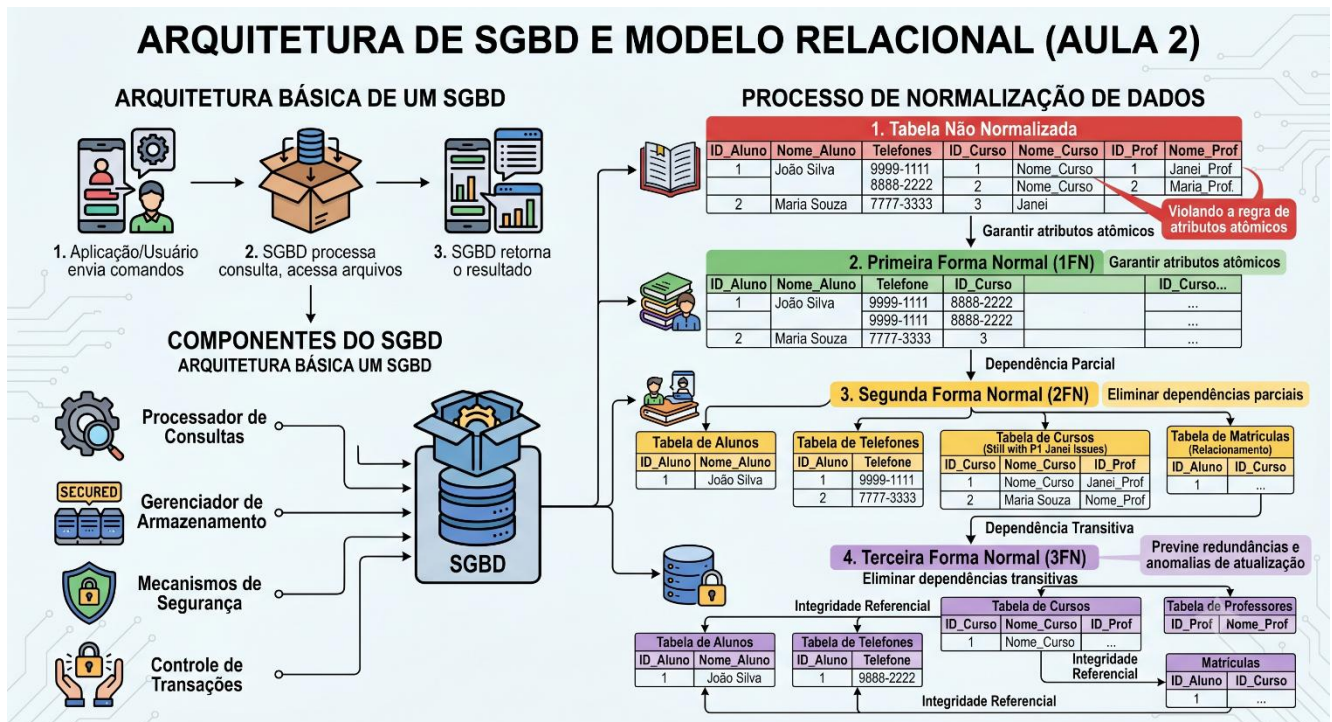
Neste curso, nosso foco será dominar o modelo relacional. Ele é muito mais do que um marco histórico, é a espinha dorsal de quase todos os sistemas corporativos, plataformas acadêmicas, aplicativos de transporte, e-commerces e APIs modernas que você irá construir e gerenciar na sua carreira como desenvolvedor Full Stack.

AULA 2 — ARQUITETURA DE SGBD E MODELO RELACIONAL

Arquitetura básica de um SGBD

De forma simplificada, uma aplicação ou usuário envia comandos ao SGBD, que processa a consulta, acessa os arquivos de dados e retorna o resultado. Nessa estrutura há componentes como processador de consultas, gerenciador de armazenamento, mecanismos de segurança e controle de transações.

Figura 3: diagrama em camadas mostrando aplicação, SGBD e arquivos de dados.



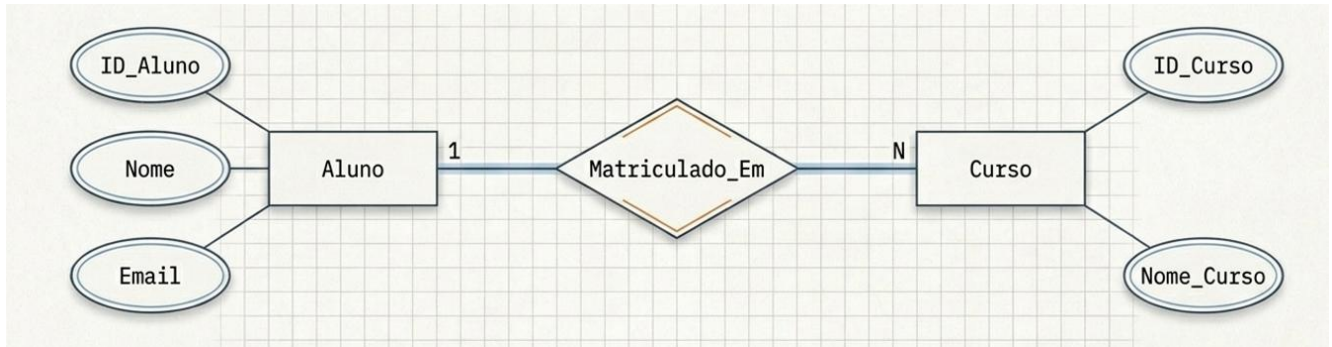
Tipos de SGBD

- MySQL: muito usado em ambientes web.
- PostgreSQL: robusto e aderente a padrões SQL.
- Oracle Database: forte presença corporativa.
- SQL Server: comum em ambientes Microsoft.

Entidades, atributos e relacionamentos

Entidade é o objeto que será representado no banco, como Aluno ou Curso. Atributo é a característica da entidade, como nome ou e-mail. Relacionamento mostra como entidades se conectam.

Figura 4: DER simples com Aluno, Curso e Matrícula.



AULA 3 — CHAVES, INTEGRIDADE, NORMALIZAÇÃO E DER

Chave primária e chave estrangeira

A chave primária identifica cada registro de uma tabela de forma única. A chave estrangeira cria relacionamento entre tabelas ao referenciar a chave primária de outra tabela.

```
CREATE TABLE alunos (  
    id_aluno INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL  
);
```

Integridade referencial

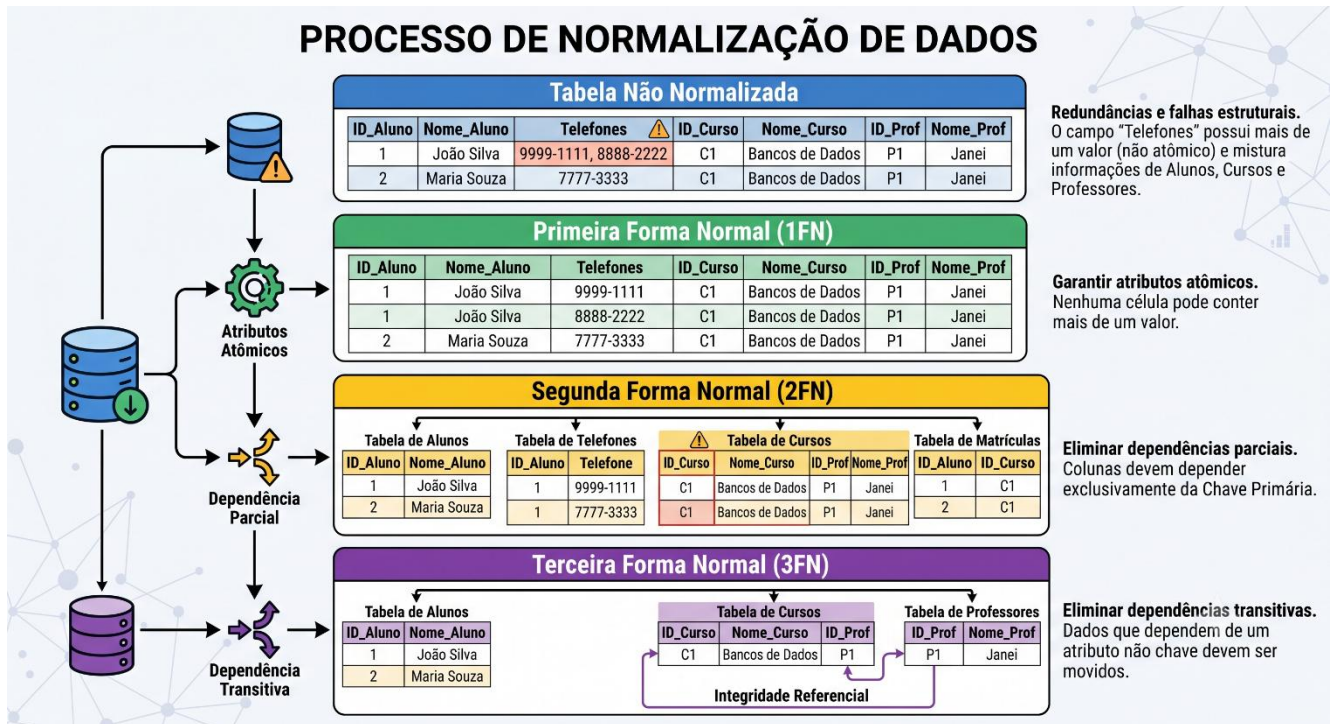
A integridade referencial impede a criação de vínculos inválidos entre tabelas. Assim, uma matrícula não deve apontar para um aluno inexistente.

Normalização

A normalização organiza os dados para reduzir redundâncias e inconsistências.

- 1FN: atributos atômicos.
- 2FN: eliminação de dependências parciais.
- 3FN: eliminação de dependências transitivas.

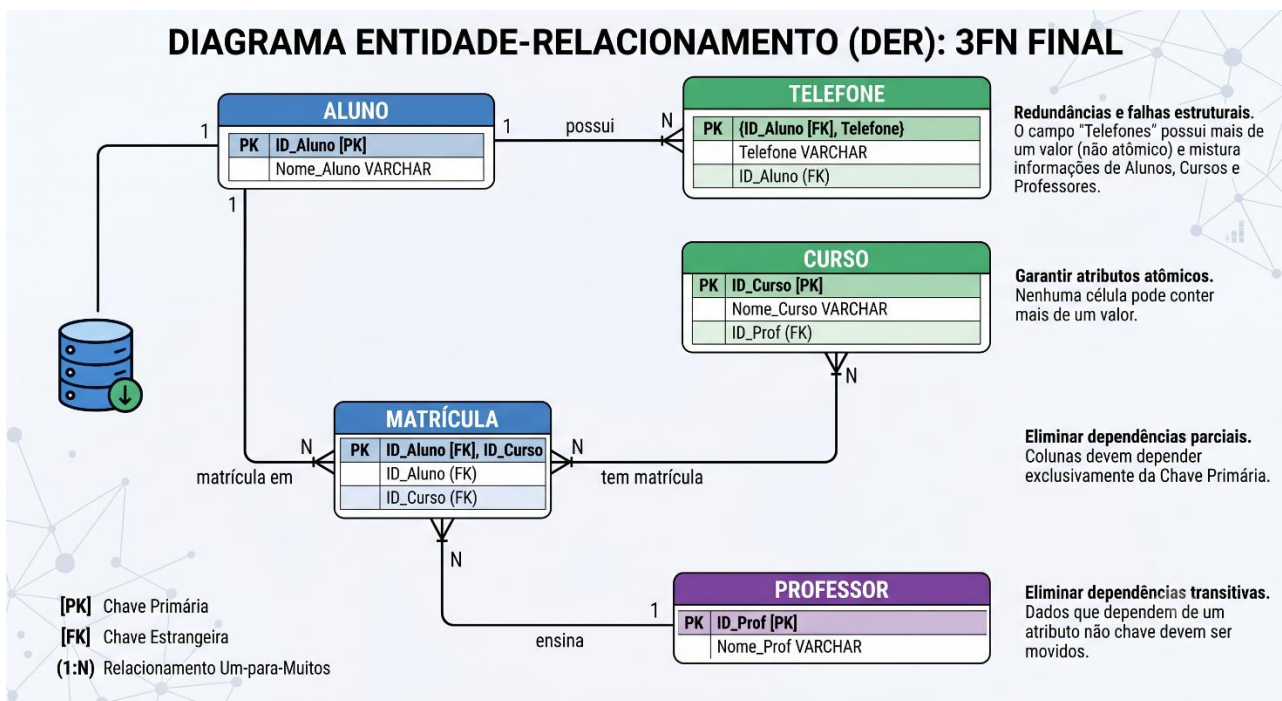
Figura 5: Tabela não normalizada sendo transformada até a 3FN.



DER

O Diagrama Entidade-Relacionamento é a representação gráfica do banco antes da implementação. Ele ajuda a visualizar cardinalidades e relacionamentos.

Figura 6: DER diagrama entidade relacionamento da 3FN



AULA 4 — SQL DDL

CREATE, ALTER e DROP

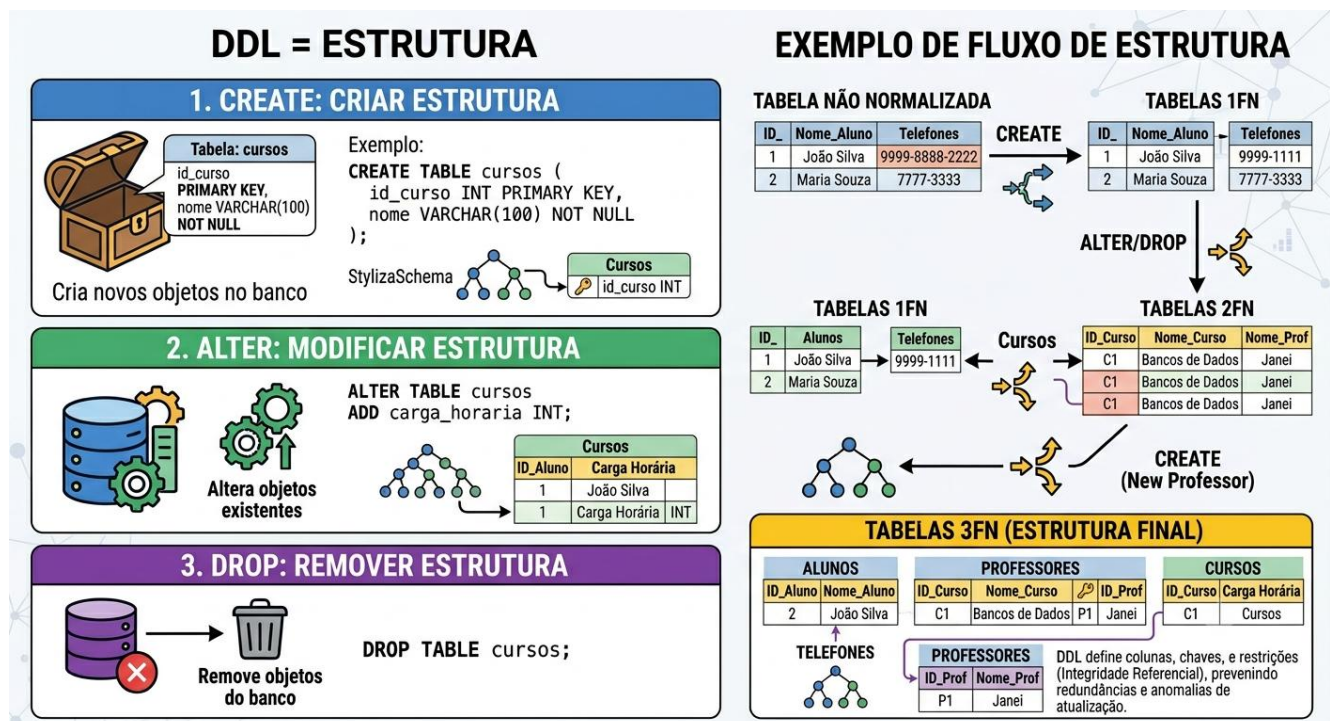
Os comandos DDL definem a estrutura do banco.

```
CREATE TABLE cursos (
  id_curso INT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL
);
```

```
ALTER TABLE cursos
ADD carga_horaria INT;
```

```
DROP TABLE cursos;
```

Figura 7: quadro-resumo com a ideia de “DDL = estrutura”.



AULA 5 — SQL DML, WHERE E ORDER BY

INSERT, SELECT, UPDATE e DELETE

Os comandos DML manipulam os dados armazenados nas tabelas.

```
INSERT INTO cursos (id_curso, nome)
VALUES (1, 'Banco de Dados');
```

```
SELECT * FROM cursos;
```

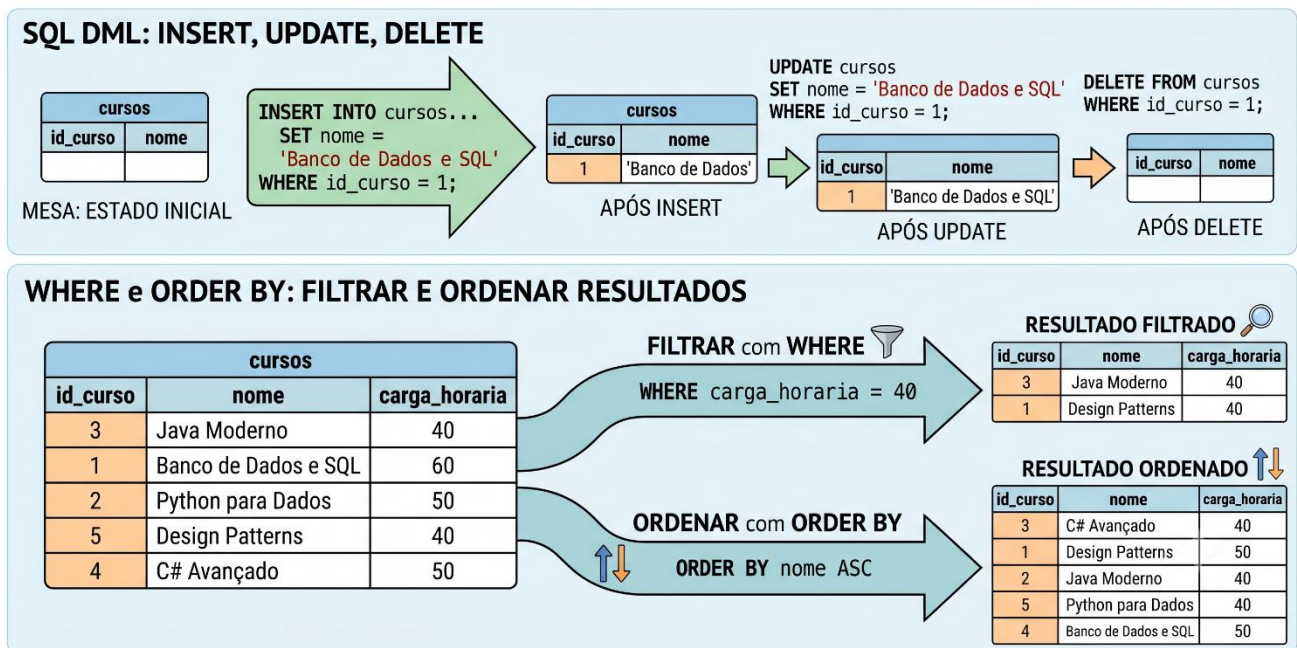
```
UPDATE cursos SET nome = 'Banco de Dados e SQL' WHERE id_curso = 1;
```

```
DELETE FROM cursos WHERE id_curso = 1;
```

WHERE e ORDER BY

WHERE filtra os registros retornados e ORDER BY organiza a ordem de exibição dos resultados.

Figura 8: Roadmap de tabela com setas mostrando filtrar e ordenar e outros exemplos.



AULA 6 — GROUP BY, HAVING E FUNÇÕES AGREGADAS

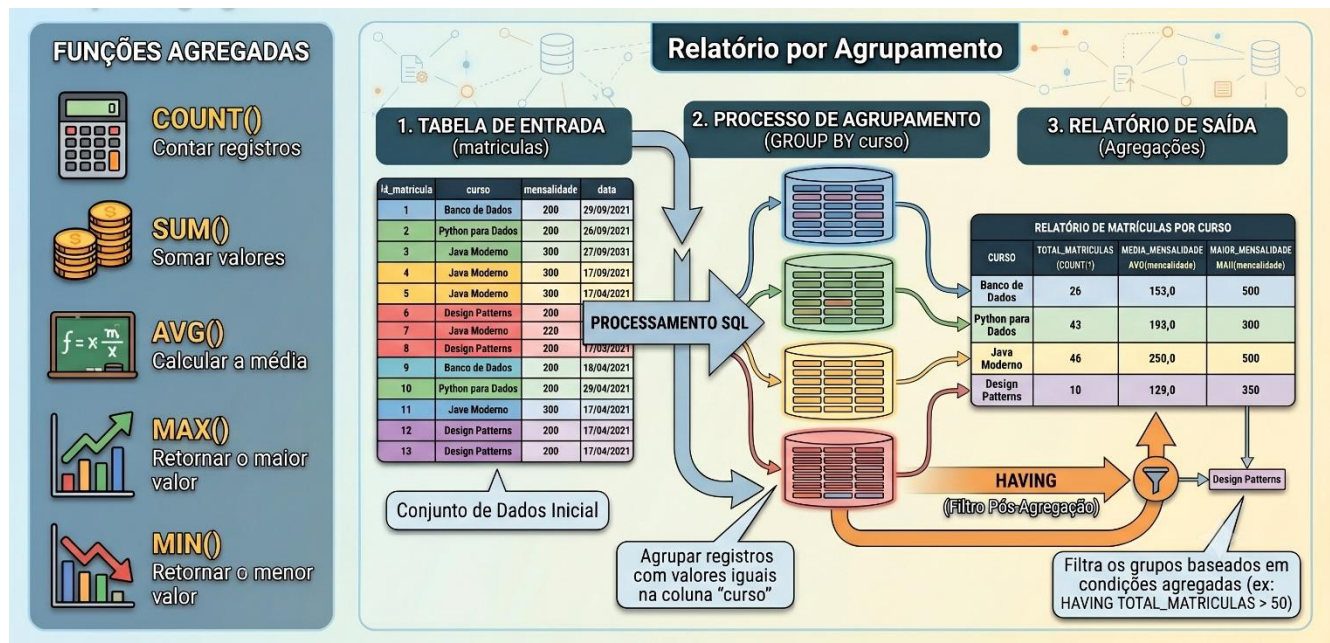
Agrupamentos e agregações

- As funções agregadas analisam conjuntos de dados.
- COUNT() conta registros.
- SUM() soma valores.
- AVG() calcula média.
- MAX() retorna maior valor.
- MIN() retorna menor valor.

```
SELECT curso, COUNT(*) AS total
FROM matriculas
GROUP BY curso;
```

HAVING é usado para filtrar grupos depois da agregação.

Figura 9: exemplo visual de relatório por agrupamento.



AULA 7 — JOINS E SUBCONSULTAS

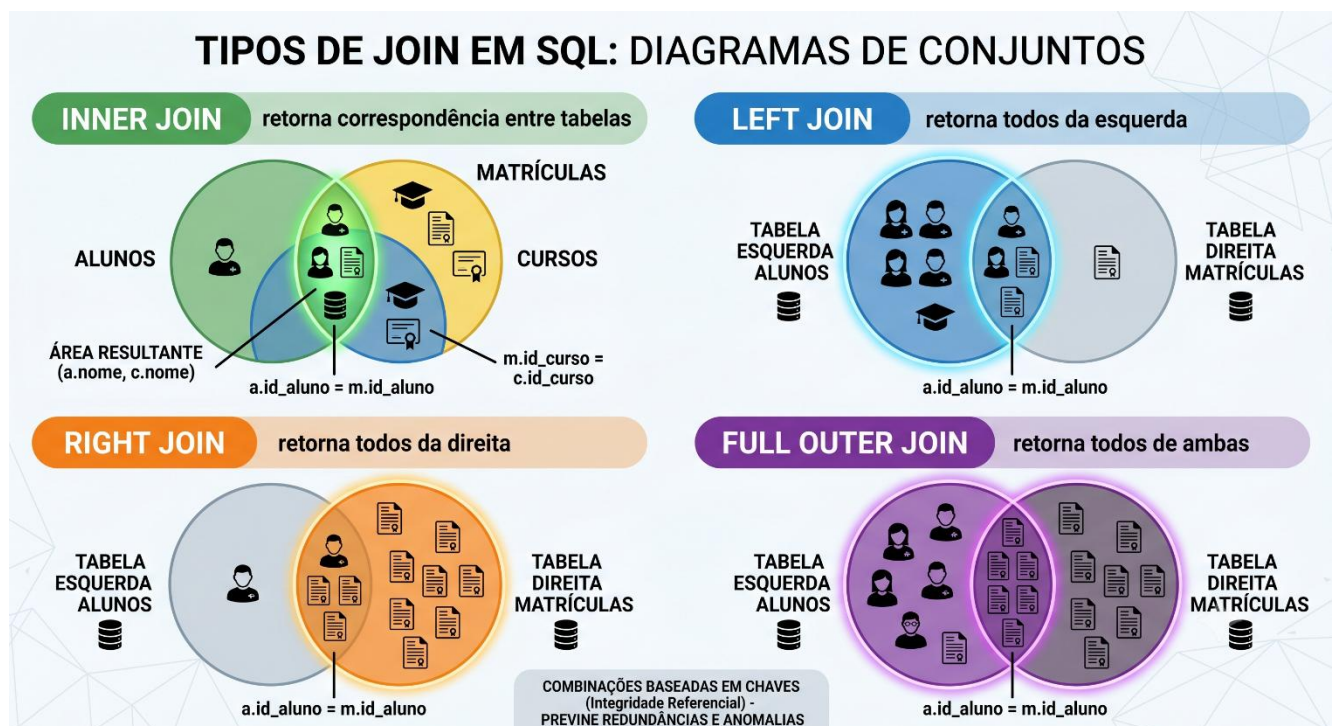
Joins

As junções combinam dados de tabelas relacionadas.

- INNER JOIN: retorna correspondência entre tabelas.
- LEFT JOIN: retorna todos da esquerda.
- RIGHT JOIN: retorna todos da direita.
- FULL OUTER JOIN: retorna todos de ambas.

```
SELECT a.nome, c.nome AS curso
FROM alunos a
INNER JOIN matriculas m ON a.id_aluno = m.id_aluno
INNER JOIN cursos c ON m.id_curso = c.id_curso;
```

Figura 10: diagramas de conjuntos para os tipos de JOIN.



Subconsultas

Subconsultas são consultas dentro de outras consultas e ajudam a resolver problemas dependentes de resultados intermediários.

AULA 8 — RECURSOS AVANÇADOS DE SQL

Views

Views são tabelas virtuais baseadas em consultas.

Índices

Índices melhoram o desempenho de busca, funcionando como o índice de um livro.

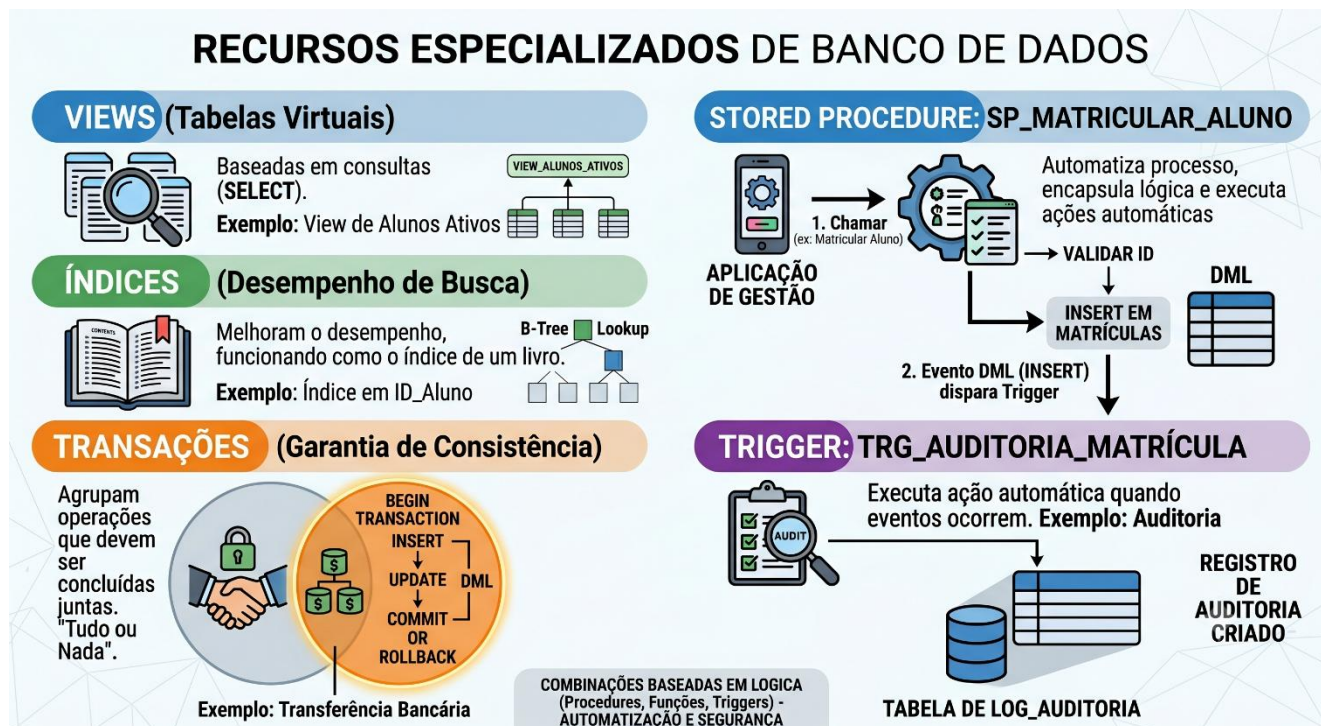
Transações

Transações garantem consistência ao agrupar operações que devem ser concluídas juntas.

Procedures, funções e triggers

Esses recursos automatizam processos, encapsulam lógica e executam ações automáticas quando eventos ocorrem.

Figura 11: fluxograma com aplicação chamando procedure e trigger executando auditoria.



AULA 9 — PROJETO PRÁTICO E REVISÃO

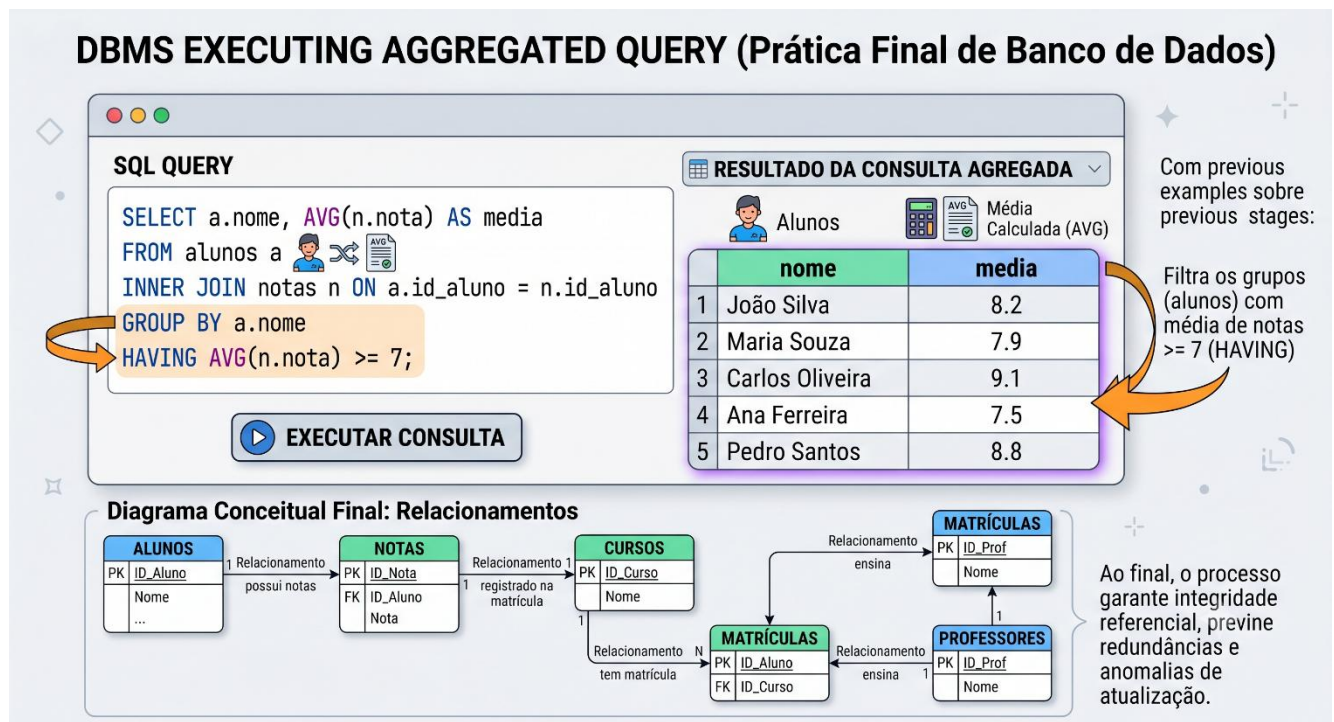
Projeto de banco relacional

A prática final deve envolver modelagem, criação de tabelas, inserção de dados e consultas mais elaboradas. Um bom exemplo é um sistema acadêmico com alunos, cursos, turmas, matrículas e notas.

Consultas complexas

```
SELECT a.nome, AVG(n.nota) AS media
FROM alunos a
INNER JOIN notas n ON a.id_aluno = n.id_aluno
GROUP BY a.nome
HAVING AVG(n.nota) >= 7;
```

Figura 12: resultado de consulta agregada no SGBD.



REFERÊNCIAS BIBLIOGRÁFICAS

Bibliografia básica

- ELMASRI, Ramez; NAVATHE, Shamkant B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson.
- SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. *Sistema de banco de dados*. 6. ed. Rio de Janeiro: Elsevier.
- DATE, C. J. *Introdução a sistemas de bancos de dados*. 8. ed. Rio de Janeiro: Elsevier.

Bibliografia complementar

- MOLINARO, Anthony. *SQL Cookbook*. Sebastopol: O'Reilly.
- BEAULIEU, Alan. *Learning SQL*. Sebastopol: O'Reilly.
- RAMAKRISHNAN, Raghu; GEHRKE, Johannes. *Database Management Systems*. 3. ed. Boston: McGraw-Hill.
- POSTGRESQL GLOBAL DEVELOPMENT GROUP. Documentação oficial do PostgreSQL. Disponível em: <https://www.postgresql.org/docs/>.
- ORACLE. MySQL Reference Manual. Disponível em: <https://dev.mysql.com/doc/>.